ICPE 2026
17th ACM/SPEC International Conference on Performance Engineering

Evaluating Kubernetes Performance for GenAI Inference

From Automatic Speech Recognition to LLM Summarization

Sai Sindhur Malleni • Raúl Sevilla • Aleksei Vasilevskii • José Castillo Lema • André Bauer

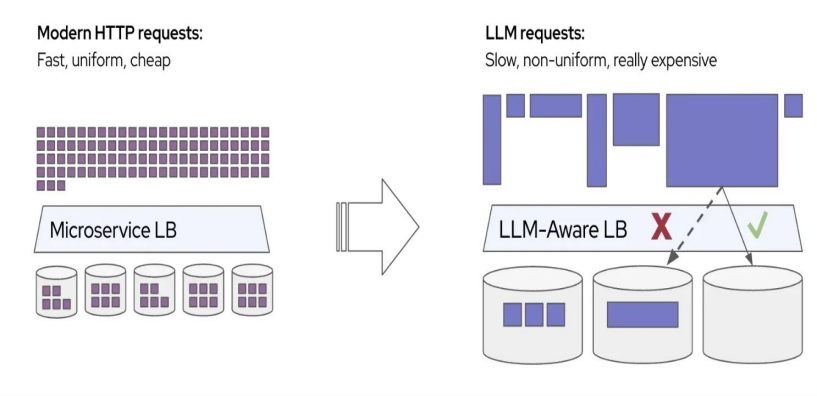
Introduction

Market Context: AI Infrastructure Surge

Big Tech (Microsoft, AWS, Google, Meta) projected to spend $725 Billion on AI infrastructure in 2026.

This 77% YoY increase underscores the massive demand for real-time inference delivered through AI factories

The Inference Challenge



Source: https://llm-d.ai/blog/llm-d-announce

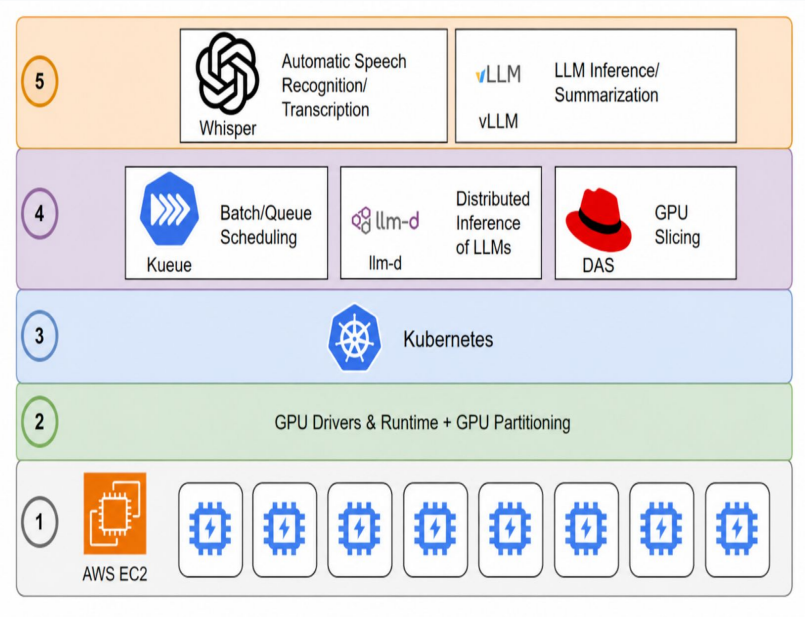
Evaluation Use Case

We evaluate a multi-stage use case on Kubernetes for transcribing earnings calls and summarizing them.

Pipeline Components:

1. Automatic Speech Recognition (Whisper)
2. LLM-based Summarization using llm-d for distributed inference on Kubernetes

Overview



Kueue

Hierarchical queuing (Local/ClusterQueues) for fair resource allocation and admission control of batch workloads.

DAS

Dynamic Accelerator Slicer provides just-in-time GPU partitioning based on actual resource requirements.

GAIE & llm-d

Extends Gateway API for latency-aware, stateful routing and prefix-cache awareness in LLM serving.

Experimental Setup

Metric	Count	Mean [s]	SD [s]	Median [s]	Min [s]	Max [s]
All Jobs	32	3127.38	1217.90	3287.51	1258.34	7405.44
Large Model Jobs	16	3077.63	1584.01	3119.85	1258.34	7405.44
Medium Model Jobs	16	3177.13	742.37	3287.51	1682.72	4380.08

Dataset Statistics. All values denote run time of the audio files to be transcribed.

Environment & Infrastructure

Red Hat OpenShift Service on AWS (ROSA) v4.19.16

- **Cluster:** 3 control (m5.2xlarge) & 3 infra (r5.xlarge) nodes
- **GPU:** 1 p4d.24xlarge node with 8 NVIDIA A100/40GB GPUs
- **Software:** NVIDIA GPU Operator v25.3.4
- **Tooling:** kube-burner, GuideLLM

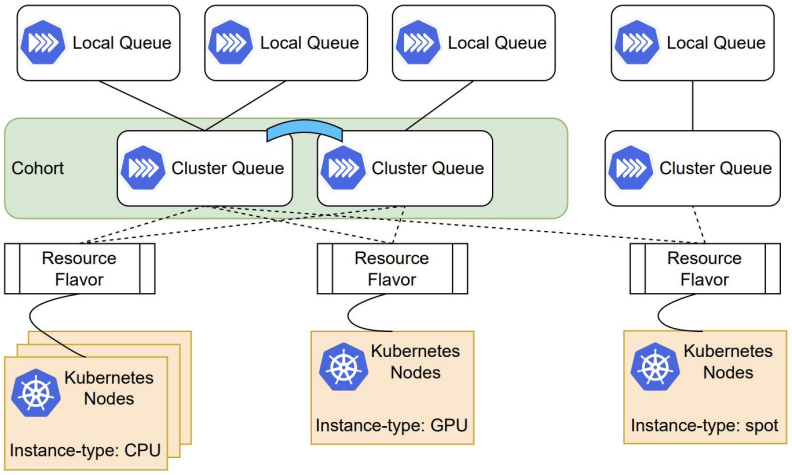
Earnings FY22 Dataset Details

125 financial earnings call transcripts from global companies

- **Subset:** 32 files selected randomly
- **Whisper Medium:** (769M params)- 16 audio files fed to a medium model
- **Whisper Large:** (1550M params) - 16 audio files fed to a large model

https://github.com/RedHatResearch/icpe26-k8s-ai-apis

Kueue - Overview



Hierarchical mapping (LocalQueue → ClusterQueue → ResourceFlavor) forming the foundation of Kueue's scheduling architecture.

Hierarchical Queuing

Kueue extends native Kubernetes scheduling with a model of LocalQueues (namespace-scoped) and ClusterQueues (global domains).

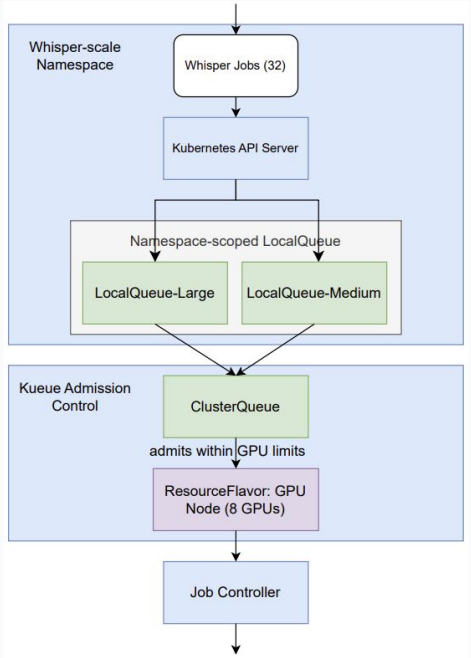
Resource Management

Introduces ResourceFlavors to represent distinct compute configurations, bridging namespace workloads with cluster-level scheduling.

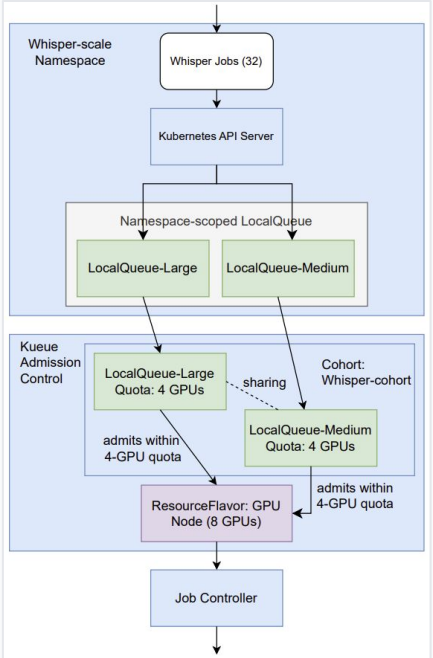
Key Benefits

Addresses job admission delay, resource fragmentation, and fairness in multi-tenant environments.

Kueue - Configuration



Single Queue



Resource Borrowing

Experimental Setups

Baseline: Native Kubernetes scheduling without Kueue.

Single Queue: BestEffortFIFO with 1 ClusterQueue.

Resource Borrowing: 2 ClusterQueues with borrowing from the collective cohort pool.

Technical Implementation

Each transcription is a Kubernetes job translating to a Kueue Workload.

Model: Whisper (entire GPU utilized).

Borrowing allows unused quota to be shared within the cohort to accommodate more concurrent jobs.

Kueue - Jobs Performance

Configuration	Makespan (Med, (Min–Max))	Queue Time (Med / P95)	Exec Time (Med)	Medium Jobs		Large Jobs	
				Queue (Med)	Exec (Med)	Queue (Med)	Exec (Med)
Pure Jobs (Baseline)	60.5 (59.1–60.5)	0.0 / 0.0	28.4	0.0	23.6	0.0	35.3
Kueue BestEffortFIFO	62.5 (62.4–65.8)	16.7 / 39.5	9.4	15.9	6.5	17.3	17.8
+ Priority	54.7 (52.2–56.1)	19.5 / 44.5	9.5	39.1	6.7	10.2	17.1
+ Preemption	51.1 (49.2–52.7)	25.0 / 41.9	8.9	36.7	6.4	3.7	15.6
2 ClusterQueues + Borrowing	55.0 (54.2–60.0)	13.7 / 32.4	8.7	10.2	6.7	25.1	18.0

All values are in minutes.

Key Trade-offs & Advantages

Predictability: Kueue ensures deterministic execution order, essential for capacity planning and SLA commitments.

Throughput: Native k8s scheduling may yield shorter makespan by chance, but Kueue guarantees fairness and reproducibility.

Optimization: Priority and preemption reduced makespan to 51.1 min and cut queue times for large jobs by half, at the expense of longer delays for medium jobs.

Borrowing Mechanism: Enabling ClusterQueues with borrowing struck the best balance, dynamically reallocating idle GPUs to improve medium job latency.

Conclusion: Combining queue management, priority, and resource sharing enables Kueue to optimize makespan while handling heterogeneous AI workloads.

Kueue - Admission Delay

Configuration	Admission Attempt Duration (ms)		
	P50	P95	P99
Kueue BestEffortFIFO	2.67	7.64	20.9
+ Priority	2.51	4.77	6.48
+ Priority + Preemption	2.57	5.48	12.8
2 LocalQueues + 2 ClusterQueues	2.82	17.0	23.7

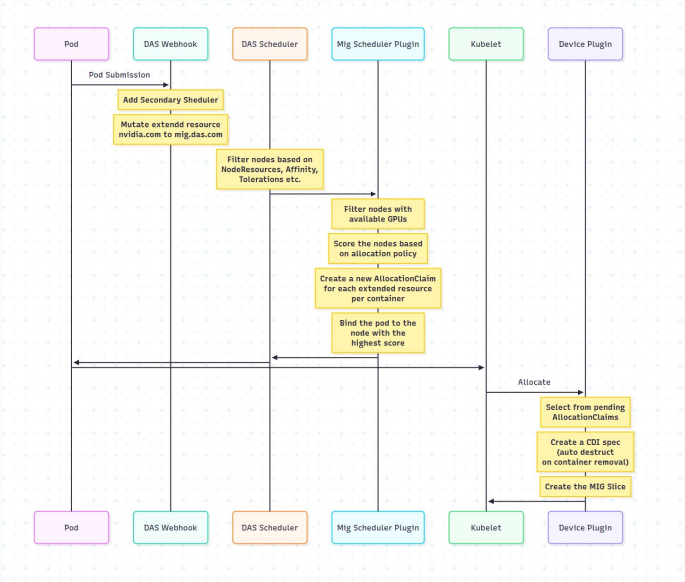
Kueue's admission control scales efficiently with sub-25ms latency across all tested configurations. Priority and preemption show minimal impact on decision times, maintaining sub-10ms results at the median. Higher tail latencies in complex multi-queue setups due to cross-queue synchronization and resource borrowing coordination remain operationally negligible compared to job execution times.

Dynamic Accelerator Slicer (DAS) - Overview

Key Challenges Addressed

- **Just-in-Time Provisioning:** Slices created only when new jobs arrive.
- **Dynamic Allocation:** Resources reclaimed immediately upon job completion.
- **Precise Configuration:** Direct coordination with NVIDIA GPU Operator.
- **Integrated Scheduling:** Binds pod scheduling with successful slice provisioning.

DAS enables multiple inference workloads to share a single GPU efficiently using NVIDIA MIG-slicing technlogy, increasing utilization and reducing idle costs.



Source: https://github.com/openshift/instaslice-operator

Conventional static pre-slicing methods may lead to fragmentation and service disruptions. DAS provides operational flexibility by dynamically responding to workload specifications.

Dynamic Accelerator Slicing - Overview

Methodology Overview

To assess DAS impact, we evaluated two configurations for Whisper-based transcription jobs:

- **With DAS:** Dynamic MIG-slicing to run concurrent jobs on a single physical GPU.
- **Without DAS:** Each job consumes an entire physical GPU.

GPU Instance Profiles:

- Medium model: 1g.5gb slice
- Large model: 3g.20gb slice

The terms 'large' and 'medium' refer to Whisper model variants, not input audio size.

A100 MIG Profiles

Config	GPC Slice #0	GPC Slice #1	GPC Slice #2	GPC Slice #3	GPC Slice #4	GPC Slice #5	GPC Slice #6
1	7						
2	4				3		
3	4				2		1
4	4				1	1	1
5	3			3			
6	3			2		1	
7	3			1	1	1	
8	2		2		3		
9	2		1	1	3		
10	1	1	2		3		
11	1	1	1	1	3		
12	2		2		2		1
13	2		1	1	2		1
14	1	1	2		2		1
15	2		1	1	1	1	1
16	1	1	2		1	1	1
17	1	1	1	1	2		1
18	1	1	1	1	1	2	
19	1	1	1	1	1	1	1

Source: https://nvidia.com/datacenter/tesla/mig-user-guide/latest/supported-mig-profiles.html

Efficiency gains are quantified by splitting GPUs into smaller slices to run multiple Whisper jobs concurrently.

DAS - Influence of MIG Slicing

Transcription Performance

Limited concurrency

Metric	No MIG	MIG
Concurrent Jobs	8	8
Mean Job Completion Time, min	14	17
Median Job Completion Time, min	10	13
P95 Job Completion Time, min	36	40
Std Dev, min	11	11
Mean GPU Utilization, %	45	19
Mean Tensor Core Utilization, %	1	1
Mean DRAM Activity, %	13	12
Mean Worker CPU Utilization, %	9	9

Key Insights

- Moderate but noticeable performance degradation on proposed MIG slice sizes.
- Resource constraints primarily impact job completion times.
- Establishes critical performance baseline for future parallel efficiency studies.
- Median completion time increases by ~30% when utilizing MIG slicing.

This setup ensured that waiting time was excluded from the job completion time measurement as both variants provided enough concurrency primitives: eight GPUs for no MIG setup and eight GPU slices for the MIG setup.

DAS - Transcription Performance

Influence of DAS

Transcription performance on the full dataset

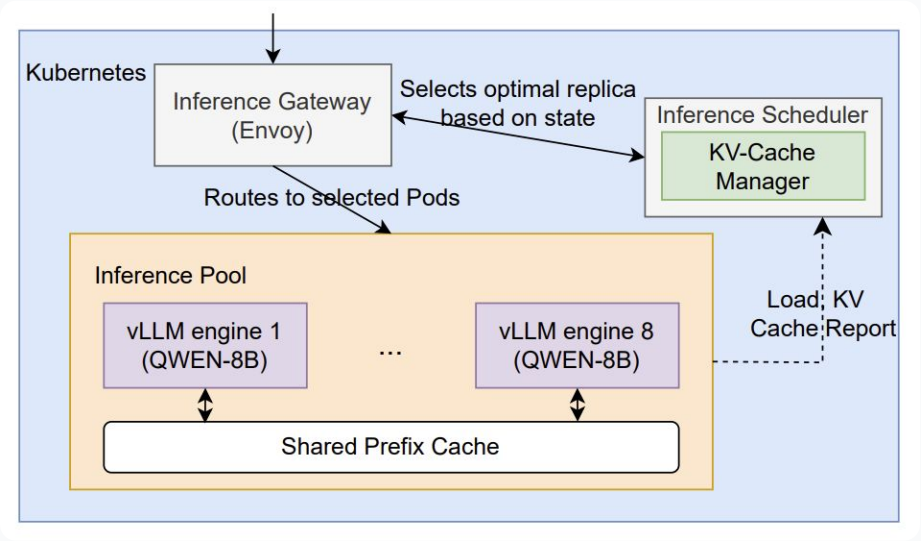
Metric	No DAS	DAS
Concurrent Jobs	8	25
Mean Job Completion Time, min	28	18
Median Job Completion Time, min	28	16
P95 Job Completion Time, min	51	36
Std Dev, min	14	9
Mean GPU Utilization, %	44	38
Mean Tensor Core Utilization, %	1	3
Mean DRAM Activity, %	13	38
Mean Worker CPU Utilization, %	10	37

Key Insights

- Parallel execution via DAS significantly reduces total completion times.
- Concurrency gains (25 vs 8 running jobs) outweigh per-job degradation on MIG slices.
- Increased execution parallelism also resulted in increased resource utilization.

The moderate decrease of the average GPU utilization is due to the nature of MIG-slicing: with certain slice allocation patterns, the residual capacity may not be allocable, resulting in lower overall utilization average.

Distributed Inference with llm-d



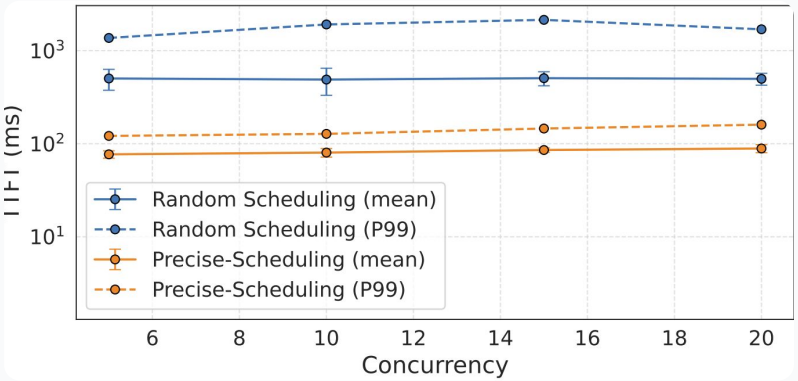
System Architecture & Methodology

To evaluate distributed inference performance, we utilized **llm-d** and the **InferenceGateway** (via GAIE).

- **Intelligent Scheduling:** Precise Prefix cache aware (Precise Scheduling) “well-lit path”.
- **Deployment:** Eight replicas of vLLM serving the Qwen/Qwen3-8B model.
- **Gateway Service:** OpenShift Service Mesh Envoy proxy acting as the GAIE gateway.

Distributed Inference - Latency Analysis

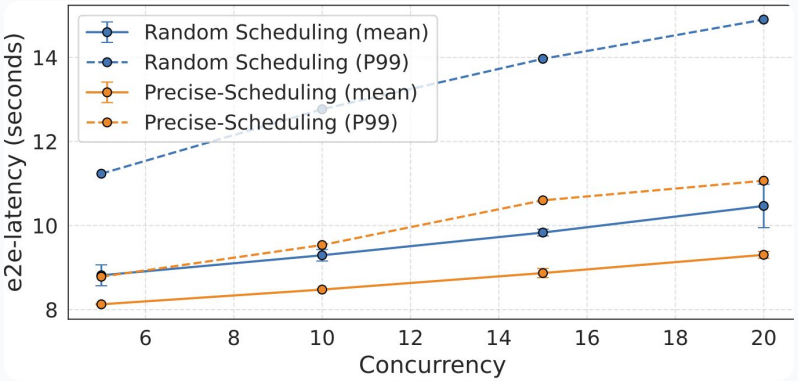
Time to First Token (TTFT)



TTFT with Random Scheduling is **~6.25x slower** on average, with a much wider 95% confidence spread (100ms vs 6ms).

The 99th percentile TTFT is up to **20x slower** than Precise Scheduling.

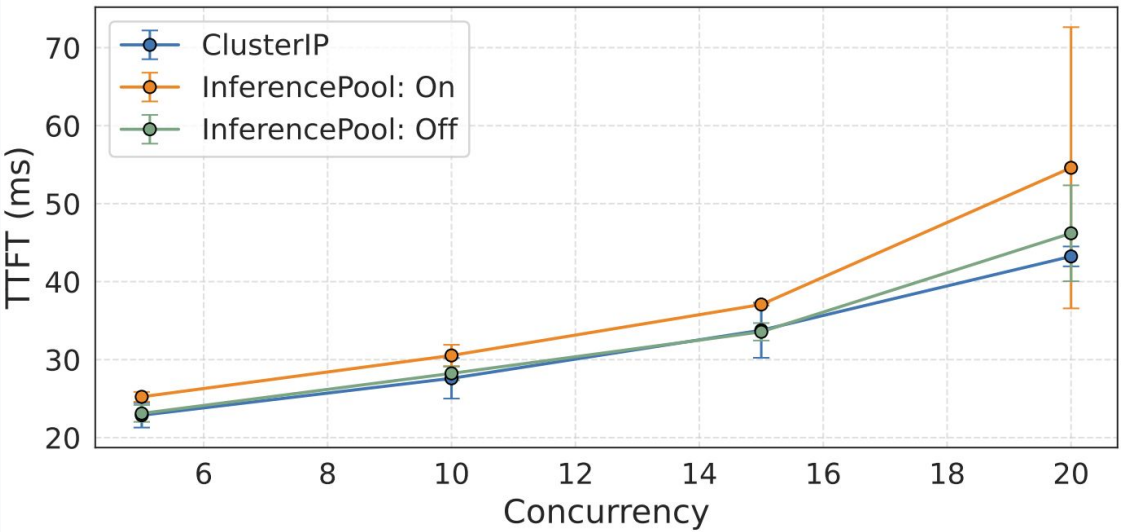
E2E Latency



Random scheduling incurs **~1s more** e2e-latency on average compared to Precise Scheduling.

At the 99th percentile, Random Scheduling is up to **6s slower** with significantly higher variability.

Distributed Inference - InferencePool Overhead



To avoid potential latency introduced by the LLM runtime and accurately measure overhead, a single replica of the vLLM Simulator pod was used instead of a real vLLM instance in this experiment.

TTFT increases with concurrency, with InferencePool showing the highest latency at scale due to gateway and endpoint picker routing overhead. This overhead is offset by significant latency improvements in real-world LLM use cases.

Takeaways

The results collectively demonstrate the indispensable nature of Kubernetes and complementary strengths of **Kueue, DAS, and distributed inference scheduling (via GAIE)** for distributed inference.

Kueue

Deterministic, fair, and reproducible scheduling.

- Negligible admission latency
- **Reduces makespan by up to 15%**
- Improves throughput for large jobs

DAS

Dynamic GPU partitioning into smaller slices.

- **36% reduction in mean job completion time**
- Enhanced parallelism
- Balanced resource sharing

GAIE + llm-d

Intelligent inference routing with Precise Scheduling.

- **90% lower 99th percentile TTFT**
- **25% lower E2E latency**
- Stable, predictable performance